

Most MCP servers are empty

Seven months of production MCP at a 350-person company

David Golverdingen · Senior Engineer & MCP Architect, Warmtebouw
MCPCon Europe · 18 September 2026

Someone who has never opened our ERP
is going to ask it about **some data**.

What is going to tell the agent what it means?

✓ Transport

solved for everyone

✓ Auth

solved for everyone

? Meaning

different at every company,
every tool, every field

“Does this building have an energy label?”

/mcpMinimal · thin

```
description  no note on null
response     energielabel: null
```



could mean ?

- no label
- not registered
- not applicable
- not loaded yet

MODEL GUESS

“This building has **no energy label.**”

/mcp · rich

```
description  INTERPRETATION: null = not registered
response     energielabel: null
```



means

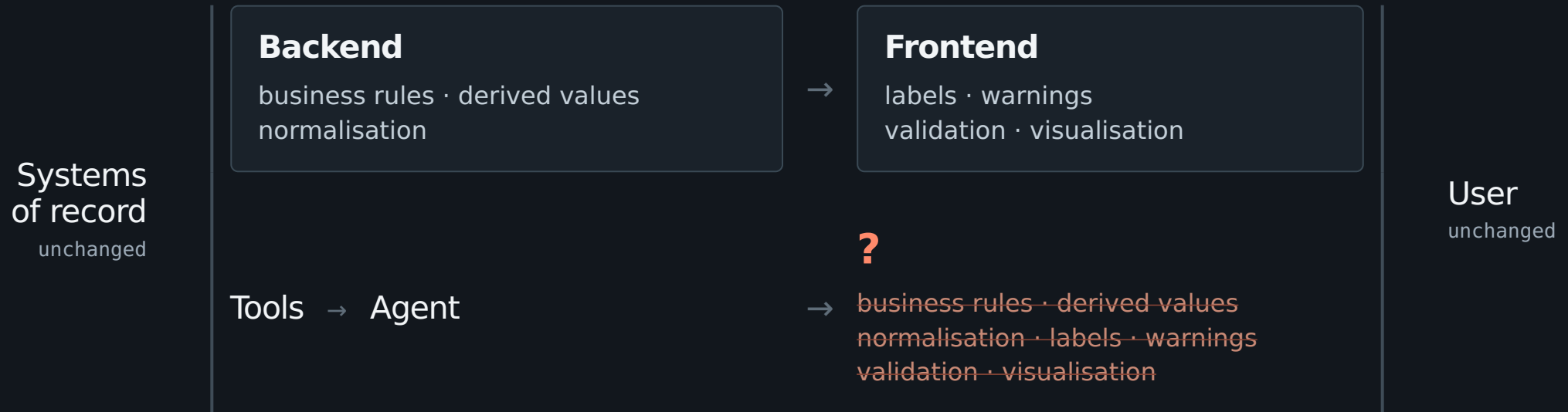
not registered

GROUNDED

“**None is registered** for this address.”

Same null. **Nothing in the response says which.**

Where did the meaning go?

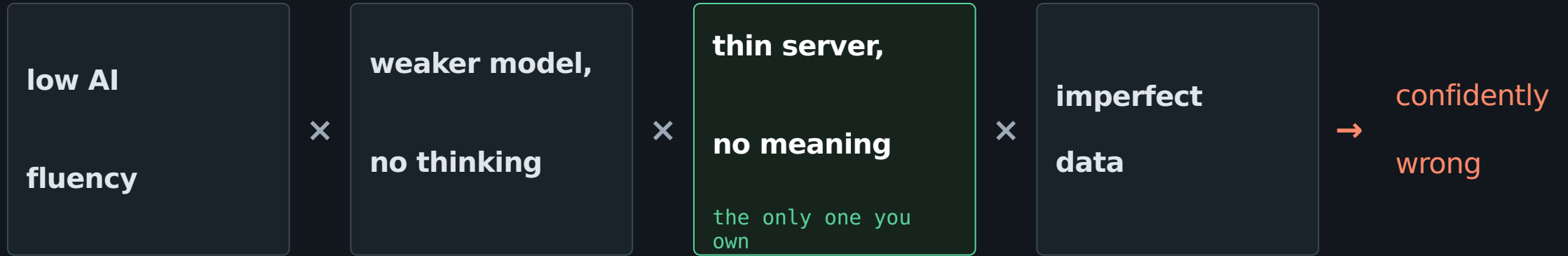


The API could be dumb **because the application was smart.**

The agent isn't empty.
The server is.

“It works” may simply mean **the model guessed correctly.**

Why it works for you, and not for them



pilots call this the error chain

Break **one** link and the odds drop.

WARMTEBOUW.



12

custom MCP servers in production, mostly non-developers

97

tools we wrote the descriptions for

8

MCP Apps that render the answers

ERP

BIM

Estimating

Building automation

Energy

External registers

IT

350-person Dutch mechanical
building services contractor.
Five developers.

We scaled capabilities, not agents



If the interface is empty, **an agent on top of it is a guess with a job title.**

Who discovers the meaning?

THIN	L1 API wrapper	nobody. The agent reconstructs the domain itself
	L2 Descriptive tool	the author, briefly
	L3 Domain-aware	human in the lead
	L4 Self-teaching	the agent, from the real data
RICH DOMAIN	THEN: PUTTING BACK THE REST OF WHAT THE APPLICATION DID	
	L5 Interactive	how it is shown · MCP Apps
	L6 Safe write	how it is changed · guarded mutation

Level 4 is not more work than level 3.
It's less, and better grounded.

Introspective Context Engineering for MCP

SCAFFOLD	ship something thin, straight from the API docs	
3 to 4 times ↻	Examine	interrogate your own deployed tool, in a fresh context
	Flag	every finding carries a confidence level
	Validate	the agent settles what it can against the data
	Encode	into the channel the model actually reads, then redeploy
	Iterate	back to Examine. In production, telemetry picks the next gap
after the loop	Validate	the expert. Only what the agent could not settle, then one more pass
HARDEN	tests, a findings log, once the loop stabilises	

The spec gives you the shape.
The data gives you the meaning.

Every tool, same eight blocks

before the call

WHEN TO USE · WHEN NOT TO USE · RELATED TOOLS

calling it

QUERY STRATEGY · RETURNS

after the answer

INTERPRETATION · ALERTS

and one more

FEEDBACK

a fixed template we grew into, not a design we started with

Three of those groups are needed at **three different moments**.

How do I know it's right?

Expert validation

the AI interviews the expert
on what it flagged ~10% of the metadata

Full review

the expert reads the finished text,
end to end

Field validation

the people whose data it is

You review. You don't author.

Every call carries what it was **trying to find out**

“fault reports on the air handling unit, to judge whether the failures recur”

“whether a quote was raised after the inspection for the defective boiler”

“counting the subscription lines before fetching the full records”

“searching on description, because the project number returned no match”

“checking whether the create committed, before retrying, to avoid a duplicate”

“redacted (special-category server)”

one field: `queryIntent`

A log of calls becomes
a log of questions.

I built `get_ticket` to return a ticket

read the whole ticket, at work-start	~ 23%
one section · one field · acceptance criteria	~ 26%
status & progress	~ 19%
verify a previous write	~ 16%
duplicate / scope triage	~ 11%
<code>queryIntent</code> , measured	

They only asked for the whole thing
because the whole thing was all I offered.

- **Selective retrieval**

one ticket, read twice, 24 seconds apart: 4 KB vs 265 KB. 67× cheaper, same information.

- **Summaries**

the server computes them, instead of shipping rows for the model to add up.

- **Alerts**

fire only when the condition applies. Conditional meaning, at zero catalog cost.

- **Derived values**

the meaning arrives as a value, not as a warning.

- **The write reports back**

16% of calls were verifying a previous write. Now the write says what it did, and the call is gone.

Not prompt tuning. **Interface engineering.**

Not ideal.
But it arrives.

	BOUND TO THE TOOL	REACHES THE MODEL
description + input schema	✓	before the call
the response payload	✓	after the call
the output schema	✓	not as model context
server instructions	✗ server-wide	client-dependent
a skill · a resource	✗ nothing ties it	only if something fetches it

Bound is not the same as delivered.
Today I have to choose where to lose.

One string. Three moments.

WHY / WHY NOT to use this

before selection

HOW to call it, what matters

after selection

WHAT the result means

after the response

**If the agent must know something before it can safely use a tool,
should **discovering** that knowledge be optional?**

binding fixes the staleness · automatic activation fixes the delivery

Most MCP servers are empty of meaning.

"What is going to tell the agent what it means?"

The interface has to.

An extracted demo repo, showing part of this:

a skill that runs the loop on **your** server

the practitioner paper

example code

a thin and a rich MCP server on the same public API

these slides, as a PDF

`davidgolverdingen.nl`

`linkedin.com/in/davidgolverdingen`



`github.com/DaveGold/
mcp-metadata-demo`